

React JS: Mastery Part 4

Side Effects and the useEffect Hook

1. What are "Side Effects"?

In React, a "Side Effect" is anything that happens outside the normal flow of rendering a component. Examples include:

- Fetching data from an API.
- Manually changing the DOM (like changing the document title).
- Setting up a subscription or a timer (setInterval).

The `useEffect` hook allows you to perform these actions in functional components.

2. Basic Syntax

```
import React, { useEffect, useState } from 'react';

function Example() {
  useEffect(() => {
    // This code runs AFTER the component renders
    console.log("Component mounted or updated!");
  });

  return <h1>Check the console</h1>;
}
```

3. The Dependency Array (Crucial)

The second argument to `useEffect` is an array. This tells React **when** to run the effect.

- **No Array**: Runs on every single render. (Rarely used).
- **Empty Array []**: Runs only **once**, when the component first appears (Mounting). Perfect for API calls.
- **With Variables [count]**: Runs only when the specified variable changes.

```
useEffect(() => {  
  fetchData();  
}, []); // Empty array = run once
```

4. The Cleanup Function

Sometimes you need to "clean up" after an effect (like stopping a timer). You do this by **returning a function** inside `useEffect`.

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Tick");  
  }, 1000);  
  
  // Cleanup function: runs when component unmounts  
  return () => clearInterval(timer);  
}, []);
```

Analogy: If `useState` is the component's memory, `useEffect` is the component's **to-do list** that it checks after every render.

5. Summary

- Use `useEffect` for API calls and timers.
- Always consider what goes in the dependency array to prevent infinite loops.
- Use the return function to prevent memory leaks (Cleanup).